



Wykład: 11

Struktury, unie, pola bitowe



Struktury

Struktury to złożone typy danych pozwalające przechowywać dane różnego typu w jednym obiekcie.

- ✓ Za pomocą struktur możliwe jest grupowanie wielu zmiennych o różnych typach.
- ✓ Za pomocą struktur można w prosty sposób organizować zbiory danych, bez konieczności korzystania z tablic.

Struktura nazywana jest też **rekordem** (szczególnie w odniesieniu do baz danych).

Deklaracja struktury w C++

Struktury tworzymy słowem kluczowym **struct**.

1. podajemy nazwę typu,
2. w nawiasie klamrowym definiujemy elementy składowe

```
struct nazwa{  
    typ nazwa_elementu;  
    typ nazwa_drugiego_elementu;  
    typ nazwa_trzeciego_elementu;  
    //...  
};
```

Uwaga!

Tak opisana struktura nie jest jeszcze egzemplarzem zmiennej a dopiero definicją nowego typu zmiennej złożonej.

Deklaracja struktury w C++

Przykład – struktura zawierająca rekord prostej bazy danych:

```
struct osoba {  
    string imie;  
    string nazwisko;  
    int wiek;  
};
```

Opisujemy typ strukturalny o nazwie „osoba”

```
osoba pracownik1, pracownik2;
```

Druga metoda:

```
struct osoba {  
    string imie;  
    string nazwisko;  
    int wiek;  
} pracownik1, pracownik2;
```

Definiujemy dwie zmienne opisanego wyżej typu o nazwach „pracownik1” i „pracownik2”

Inicjalizacja struktury

Struktury można inicjalizować już w chwili ich tworzenia.

```
struct osoba {  
    string imie;  
    string nazwisko;  
    int wiek;  
};  
osoba ktos = {"Jan", "Kowalski", 10};
```

Lub też krócej:

```
struct osoba {  
    string imie;  
    string nazwisko;  
    int wiek;  
} ktos = {"Jan", "Kowalski", 10};
```

Zapis i odczyt danych struktury



```

1  #include <iostream>
2  #include <cmath>
3
4  using namespace std;
5
6  class Zespolona {
7  private:
8      double re;
9      double im;
10
11 public:
12     // konstruktor domyślny
13     Zespolona() {
14         re = 0;
15         im = 0;
16     }
17
18     // konstruktor z parametrami
19     Zespolona(double x, double y) {
20         re = x;
21         im = y;
22     }
23
24     // konstruktor kopiujący
25     Zespolona(const Zespolona& z) {
26         re = z.re;
27         im = z.im;
28     }
29
30     // operator +
31     Zespolona operator+(const Zespolona& z) {
32         return Zespolona(re + z.re, im + z.im);
33     }
34
35     // operator *
36     friend Zespolona operator*(const Zespolona& a, const Zespolona& b) {
37         double nowyRe = a.re * b.re - a.im * b.im;
38         double nowyIm = a.re * b.im + a.im * b.re;
39
40         return Zespolona(nowyRe, nowyIm);
41     }
42
43     // operator ==
44     bool operator==(const Zespolona& z) {
45         return re == z.re && im == z.im;
46     }
47
48     // operator !=
49     bool operator!=(const Zespolona& z) {
50         return !(*this == z);
51     }
52

```

```

53     // operator <<
54     friend ostream& operator<<(ostream& out, const Zespolona& z) {
55         out << z.re << " + " << z.im << "i";
56         return out;
57     }
58
59     // gettery
60     double getRe() {
61         return re;
62     }
63     double getIm() {
64         return im;
65     }
66 };
67
68 // klasa potomna
69 class Zespolona_V2 : public Zespolona {
70 public:
71     Zespolona_V2(double x, double y)
72         : Zespolona(x, y) {}
73
74     // moduł liczby zespolonej
75     double getModul() {
76         return sqrt(
77             getRe() * getRe() +
78             getIm() * getIm()
79         );
80     }
81
82     // liczba sprzężona
83     Zespolona getSprzezona() {
84         return Zespolona(
85             getRe(),
86             -getIm()
87         );
88     }
89 };
90
91 int main() {
92     Zespolona a(2, 3);
93     Zespolona b(1, 4);
94
95     Zespolona suma = a + b;
96     Zespolona iloczyn = a * b;
97
98     cout << "A = " << a << endl;
99     cout << "B = " << b << endl;
100    cout << "Suma = " << suma << endl;
101    cout << "Iloczyn = " << iloczyn << endl;
102    if (a == b)
103        cout << "Rowne" << endl;
104    else
105        cout << "Rozne" << endl;
106
107    Zespolona_V2 x(3, 4);
108    cout << "Modul = " << x.getModul() << endl;
109    cout << "Sprzezona = " << x.getSprzezona() << endl;
110    return 0;
111 }

```

Zapis i odczyt danych struktury

Odczyt z pól struktury:

```
cout << ktos.imie << endl;  
cout << ktos.nazwisko << endl;  
cout << ktos.wiek;
```

```
struct osoba {  
    string imie;  
    string nazwisko;  
    int wiek;  
} ktos, ktos_inny;
```


Struktury globalne i lokalne

- ✓ Struktura stworzona przed funkcją `main()` będzie strukturą globalną, (każdy podprogram będzie mógł z niej korzystać).
- ✓ Struktura stworzona wewnątrz jakiegoś bloku, będzie lokalną i widoczna tylko w tym miejscu.

Globalna

```
6 struct osoba {  
7     string imie;  
8     string nazwisko;  
9     int wiek;  
10 } ktos;  
11  
12 int main()  
13 {  
14     return 0;  
15 }
```

Lokalna

```
7 int main()  
8 {  
9     struct osoba {  
10         string imie;  
11         string nazwisko;  
12         int wiek;  
13     } ktos;  
14     return 0;  
15 }
```

Tablice struktur

Tablicę struktur tworzymy i dowołujemy się do niej w ten sam sposób co do zwykłych tablic prostych zmiennych.

```
nazwa_struktury nazwa_tablicy [liczba_elementów];
```

Tablice możemy tworzyć też bezpośrednio po deklaracji i definicji struktury:

```
struct punkty{  
    int x, y;  
    char nazwa;  
} tab[1000];
```



```

1  #include <iostream>
2  using namespace std;
3
4  class Time {
5  private:
6      int seconds;
7
8  public:
9      // konstruktor domyślny
10     Time() {
11         seconds = 0;
12     }
13
14     // konstruktor z parametrem
15     Time(int s) {
16         seconds = s;
17     }
18
19     // konstruktor kopiujący
20     Time(const Time& t) {
21         seconds = t.seconds;
22     }
23
24     // operator +
25     Time operator+(const Time& t) {
26         return Time(seconds + t.seconds);
27     }
28
29     // operator -
30     Time operator-(const Time& t) {
31         return Time(seconds - t.seconds);
32     }
33
34     // operator *
35     friend Time operator*(const Time& t, int liczba) {
36         return Time(t.seconds * liczba);
37     }
38
39     // operator ==
40     bool operator==(const Time& t) {
41         return seconds == t.seconds;
42     }
43
44     // operator !=
45     bool operator!=(const Time& t) {
46         return seconds != t.seconds;
47     }
48
49     // operator <<
50     friend ostream& operator<<(ostream& out, const Time& t) {
51         out << t.seconds << " sekund";
52         return out;
53     }

```

```

54     // getter
55     int getTotalSeconds() {
56         return seconds;
57     }
58 };
59
60 // klasa potomna
61 class Day : public Time {
62 public:
63     Day(int s)
64         : Time(s) {}
65
66     int getSeconds() {
67         return getTotalSeconds() % 60;
68     }
69
70     int getMinutes() {
71         return (getTotalSeconds() / 60) % 60;
72     }
73
74     int getTotalMinutes() {
75         return getTotalSeconds() / 60;
76     }
77
78     int getHours() {
79         return (getTotalSeconds() / 3600) % 24;
80     }
81
82     int getTotalHours() {
83         return getTotalSeconds() / 3600;
84     }
85
86     int getDays() {
87         return getTotalSeconds() / 86400;
88     }
89 };
90
91 int main() {
92     Time a(120);
93     Time b(30);
94
95     Time suma = a + b;
96     Time roznica = a - b;
97     Time mnozenie = a * 2;
98
99     cout << "A = " << a << endl;
100    cout << "B = " << b << endl;
101    cout << "Suma = " << suma << endl;
102    cout << "Roznica = " << roznica << endl;
103    cout << "Mnozenie = " << mnozenie << endl;

```

```

104    if (a == b)
105        cout << "Rowne" << endl;
106    else
107        cout << "Rozne" << endl;
108
109    Day d(90061);
110
111    cout << "Sekundy = " << d.getSeconds() << endl;
112    cout << "Minuty = " << d.getMinutes() << endl;
113    cout << "Godziny = " << d.getHours() << endl;
114    cout << "Dni = " << d.getDays() << endl;
115
116    return 0;
117 }

```

Zagnieżdżenie struktur

Zagnieżdżanie struktur polega na deklarowaniu pól jednej struktury jako typ strukturalny innej struktury.

- ✓ Struktury można zagnieżdżać wielokrotnie
- ✓ Wiele typów strukturalnych używać można jednocześnie jako pól jednej struktury,

```
5 struct adres{  
6     string miejscowosc;  
7     string ulica;  
8     int nr_domu;  
9 };  
10  
11 struct student{  
12     string imie;  
13     string nazwisko;  
14     adres dom;  
15 };
```

Zagnieżdżenie struktur

- ✓ Do pól zagnieżdżonych struktur odwołujemy wykorzystując wielokrotnie operator "."

```
19 student kowalski;  
20 |  
21 cin>>kowalski.imie;  
22 cin>>kowalski.nazwisko;  
23 cin>>kowalski.dom.miejscowosc;  
24 cin>>kowalski.dom.nr_domu;  
25  
26 cout<<kowalski.imie<<endl;  
27 cout<<kowalski.nazwisko<<endl;  
28 cout<<kowalski.dom.miejscowosc<<endl;  
29 cout<<kowalski.dom.nr_domu<<endl;
```

Struktury jako wartość funkcji

Funkcja może zwracać zmienną typu strukturalnego.

```
5 struct rgb{  
6     int r;  
7     int g;  
8     int b;  
9 };
```

```
11 rgb kolor()  
12 {  
13     rgb pom;  
14     pom.r=255;  
15     pom.g=0;  
16     pom.b=0;  
17     return pom;  
18 }
```

```
20 int main()  
21 {  
22     rgb wynik;  
23     wynik=kolor();  
24     cout<<wynik.r << wynik.g << wynik.b;  
25     return 0;  
26 }
```

Możemy więc zapakować do niej kilka zmiennych typu prostego

Struktury jako wartość funkcji



```

1  #include <iostream>
2  #include <cmath>
3
4  using namespace std;
5
6  class Wektor2D {
7  private:
8      double x;
9      double y;
10
11  public:
12      // konstruktor domyślny
13      Wektor2D() {
14          x = 0;
15          y = 0;
16      }
17
18      // konstruktor z parametrami
19      Wektor2D(double a, double b) {
20          x = a;
21          y = b;
22      }
23
24      // konstruktor kopiujący
25      Wektor2D(const Wektor2D& w) {
26          x = w.x;
27          y = w.y;
28      }
29
30      // operator +
31      Wektor2D operator+(const Wektor2D& w) {
32          return Wektor2D(x + w.x, y + w.y);
33      }
34
35      // operator *
36      friend Wektor2D operator*(const Wektor2D& w, double liczba) {
37          return Wektor2D(
38              w.x * liczba,
39              w.y * liczba
40          );
41      }
42
43      // operator ==
44      bool operator==(const Wektor2D& w) {
45          return x == w.x && y == w.y;
46      }
47
48      // operator !=
49      bool operator!=(const Wektor2D& w) {
50          return !(*this == w);
51      }

```

```

52  // operator <<
53  friend ostream& operator<<(ostream& out, const Wektor2D& w) {
54      out << "(" << w.x << ", " << w.y << ")";
55      return out;
56  }
57
58  // gettery
59  double getX() {
60      return x;
61  }
62
63  double getY() {
64      return y;
65  }
66
67  // klasa potomna
68  class Wektor2DKolor : public Wektor2D {
69  private:
70      string kolor;
71
72  public:
73      Wektor2DKolor(double a, double b, string k)
74          : Wektor2D(a, b) {
75          kolor = k;
76      }
77
78      // moduł wektora
79      double getModul() {
80          return sqrt(
81              getX() * getX() +
82              getY() * getY()
83          );
84      }
85
86      // setter koloru
87      void setKolor(string k) {
88          kolor = k;
89      }
90
91      // getter koloru
92      string getKolor() {
93          return kolor;
94      }
95
96      // operator <<
97      friend ostream& operator<<(ostream& out, const Wektor2DKolor& w) {
98          out << "("
99              << w.getX()
100              << ", "
101              << w.getY()
102              << ") kolor: "
103              << w.kolor;
104          return out;
105      }
106
107  };

```

```

109  int main() {
110
111      Wektor2D a(2, 3);
112      Wektor2D b(1, 4);
113
114      Wektor2D suma = a + b;
115      Wektor2D mnozenie = a * 2;
116
117      cout << "A = " << a << endl;
118      cout << "B = " << b << endl;
119
120      cout << "Suma = " << suma << endl;
121      cout << "Mnozenie = " << mnozenie << endl;
122
123      if (a == b)
124          cout << "Rowne" << endl;
125      else
126          cout << "Rozne" << endl;
127
128      Wektor2DKolor w(3, 4, "czerwony");
129
130      cout << "Wektor kolor = " << w << endl;
131
132      cout << "Modul = "
133          << w.getModul()
134          << endl;
135
136      return 0;
137  }

```

Podstawy programowania





Unia typem definiowanym przez użytkownika.

Od struktur różni ją to że swoje składniki zapisuje w tym samym (współdzielonym) obszarze pamięci.

Oznacza to, że w danej chwili, unia może przechowywać wartość wyłącznie **jednej** ze swoich zmiennych składowych

```
union PrzykładowaUnia
{
    int liczba_calkowita;
    char znak;
    double liczba_rzeczywista;
};
```



Jeżeli unia zawiera np. obiekt typu int i double, gdy aktualnie korzystamy z double (8B) to po odczytaniu wartości int(4B) bez uprzedniego zapisu do niej pojawią się zwykłe śmieci.

- **jednocześnie używamy tylko jednego obiektu.**



Unie

```
union nazwa{
    typ pierwszy_element;
    typ drugi_element;
    ...
    typ n_ty_element;
};

int main()
{
    //tworzenie unii
    nazwa unia;

    //odwoływanie się do elementów
    unia.pierwszy_element = 0;

    return 0;
}
```



```

1  #include <iostream>
2  using namespace std;
3
4  class KontoBankowe {
5  private:
6      double saldo;
7      string imie;
8
9  public:
10     // konstruktor domyślny
11     KontoBankowe() {
12         saldo = 0;
13         imie = "";
14     }
15
16     // konstruktor z parametrami
17     KontoBankowe(double s, string i) {
18         saldo = s;
19         imie = i;
20     }
21
22     // konstruktor kopiujący
23     KontoBankowe(const KontoBankowe& k) {
24         saldo = k.saldo;
25         imie = k.imie;
26     }
27
28     // operator +
29     KontoBankowe operator+(const KontoBankowe& k) {
30         return KontoBankowe(
31             saldo + k.saldo,
32             imie
33         );
34     }
35     // operator -
36     KontoBankowe operator-(const KontoBankowe& k) {
37         return KontoBankowe(
38             saldo - k.saldo,
39             imie
40         );
41     }
42
43     // operator +=
44     void operator+=(double kwota) {
45         saldo += kwota;
46     }
47
48     // operator -=
49     void operator-=(double kwota) {
50         saldo -= kwota;
51     }
52

```

```

53     // operator ==
54     bool operator==(const KontoBankowe& k) {
55         return saldo == k.saldo;
56     }
57
58     // operator !=
59     bool operator!=(const KontoBankowe& k) {
60         return saldo != k.saldo;
61     }
62
63     // operator <<
64     friend ostream& operator<<(ostream& out, const KontoBankowe& k) {
65         out << "Imie: "
66             << k.imie
67             << " Saldo: "
68             << k.saldo;
69         return out;
70     }
71
72     // getter
73     double getSaldo() {
74         return saldo;
75     }
76 };
77
78 // klasa potomna
79 class KontoWalutowe : public KontoBankowe {
80 private:
81     double kurs;
82
83 public:
84     KontoWalutowe(double s, string i, double k)
85         : KontoBankowe(s, i) {
86         kurs = k;
87     }
88
89     // setter kursu
90     void setKurs(double k) {
91         kurs = k;
92     }
93
94     // getter kursu
95     double getKurs() {
96         return kurs;
97     }
98
99     // funkcja zaprzyjaźniona

```

```

100     friend double Przelicz(KontoWalutowe& konto) {
101         return konto.getSaldo() * konto.kurs;
102     }
103 };
104
105 int main() {
106     KontoBankowe a(1000, "Jan");
107     KontoBankowe b(500, "Adam");
108
109     KontoBankowe suma = a + b;
110     KontoBankowe roznica = a - b;
111
112     a += 200;
113     b -= 100;
114
115     cout << "A = " << a << endl;
116     cout << "B = " << b << endl;
117
118     cout << "Suma = "
119         << suma
120         << endl;
121
122     cout << "Roznica = "
123         << roznica
124         << endl;
125
126     if (a == b)
127         cout << "Rowne" << endl;
128     else
129         cout << "Rozne" << endl;
130
131     KontoWalutowe k(1000, "Marek", 4.2);
132
133     cout << "Konto walutowe = "
134         << k
135         << endl;
136
137     cout << "Po przeliczeniu = "
138         << Przelicz(k)
139         << endl;
140
141     return 0;

```



Unie mogą być składowymi innych obiektów takich jak struktur czy

```
class  
{- union liczba{  
    int calkowita;  
    double rzeczywista;  
};  
  
{- struct samochod{  
    char marka[20];  
    char model[20];  
    int rocznik;  
    liczba pojemnosc;  
};
```

```
cout<<"Podaj pojemnosc: ";  
cin>>renault.pojemnosc.rzeczywista;
```



Pola bitowe



Oprócz zwykłych pól w strukturach możemy zastosować **pola bitowe**.

Pole bitowe to wydzielenie pewnej stałej liczby bitów na daną zmienną.

Np.: zmienna typu char zajmuje w pamięci 1 bajt = 8 bitów. Możemy ją okroić lub rozszerzyć o porcję bitów dostosowaną do potrzeb programu.

Pamięć jaką będzie zajmować takie pole będzie zawsze krotnością bajtów danego typu zmiennej. Np. jeśli stworzymy pole typu int na 4 bity, to i tak zostanie przydzielona pamięć na cały typ int czyli 4 bajty = 32 bity, a używać będziemy mogli tylko tych czterech bitów.

Więc gdzie tu oszczędność?

Gdy stworzymy na przykład pięć pól bitowych typu int, i każde będzie zajmowało 6 bitów, to w pamięci zostanie zarezerwowany obszar na jeden int, czyli 32 bity, ponieważ $5 \cdot 6 = 30 \leq 32$.



Pola te tworzymy zgodnie z zasadą:

typ nazwa : [ilość bitów];

```
int pole_bitowe : 4; //pole bitowe na czterech bitach
```


Pola bitowe

```
1  #include<iostream>
2  #include<cstdlib>
3  using namespace std;
4
5  struct pola_bitowe{
6      int a:4;
7      int b:4;
8  };
9
10 int main()
11 {
12     pola_bitowe x;
13     cout<<"Struktura zajmuje "<<sizeof(x)<<" bajty pamięci\n";
14     //cztery bajty, tyle ile jeden int
15     x.a = 7;
16     //maksymalna wartość dodatnia jaką możemy nadać dla typu int na 4 bitach
17     cout<<x.a<<endl;
18     x.a--;
19     //pola bitowe możemy inkrementować i dekrementować tak jak zmienne typu int
20     cout<<x.a<<endl;
21     return 0;
22 }
```

Literatura:

W prezentacji wykorzystano przykłady i fragmenty:

- Grębosz J. : ***Symfonia C++, Programowanie w języku C++ orientowane obiektowo***, Wydawnictwo Edition 2000.
- Jakubczyk K.: *Turbo Pascal i Borland C++ Przykłady*, Helion.

Warto zajrzeć także do:

- Sokół R. : ***Microsoft Visual Studio 2012 Programowanie w Ci C++***, Helion.
- Kernighan B. W., Ritchie D. M.: ***język ANSI C***, Wydawnictwo Naukowo Techniczne.

Dla bardziej zaawansowanych:

- Grębosz J. : ***Pasja C++***, Wydawnictwo Edition 2000.
- Meyers S.: ***język C++ bardziej efektywnie***, Wydawnictwo Naukowo Techniczne